APCS 程式設計系列 · U09 · 實作中高級 · Python版

堆疊、佇列與內建資料結構 詳解

stack 後進先出 · 括號配對與後序運算式 · queue 先進先出 · deque · 優先佇列 · map / set 計數去重 · 陣列模擬

Part A 答案速查

1	2	3	4	5	6	7	8	9	10
B	A	C	D	A	A	B	A	D	C
11	12	13							
C	B	D							

Part A 程式識讀詳解

第 1 題 答案：(B)

- i = 2 時放進 2 又馬上拿出，印 2；i = 4 時放進 4 又拿出，印 4。
- 最後堆疊裡剩 1、3（3 在上面），後進先出，依序印 3、1。

【錯】選 2 4 1 3 是把剩下的部分當成佇列（先進先出）。

第 2 題 答案：(A)

- 丟 1、把 2 移到最後；丟 3、把 4 移到最後；丟 5、把 6 移到最後，剩 2 4 6。
- 丟 2、把 4 移到最後，剩 6 4；丟 6、把 4 移到最後，最後剩 4。

第 3 題 答案：(C)

- 後序：先寫兩個運算元，再寫運算符號。3 + 4 → 3 4 +；5 - 2 → 5 2 -；兩個結果相乘，* 放最後。
- 用堆疊驗算：3 4 + 得 7，5 2 - 得 3，最後 7 * 3 = 21，與原式相同。

【錯】* + 3 4 - 5 2 是「前序」（符號寫在前面）。

第 4 題 答案：(D)

- 第一次：b = 1、a = 4，放回 4 - 1 = 3，堆疊變成 9 3。
- 第二次：b = 3、a = 9，印出 9 - 3 = 6。這就是後序運算式 9 4 1 - - 的算法。

【錯】把先拿出來的當左邊（算 b - a），就會得到負數。

第 5 題 答案：(A)

- 要先取出 3，必須先放 1、2、3；這時 2 壓在 1 上面。
- 下一個能取出的只有 2（或再放 4 進來），不可能直接拿到 1。

【錯】2 1 4 3：放 1、2，取 2、取 1，放 3、4，取 4、取 3，是可能的。

第 6 題 答案：(A)

- heapq 一律「小的先出」，與放入順序無關。
- 先拿出最小的 1，再拿出剩下最小的 3。

【錯】選 8 5 是 C++ priority_queue（大的先出）的規則。

第 7 題 答案：(B)

- pop(0) 拿掉頭的 1，pop() 拿掉尾巴的 5，剩 [2, 3, 4]。
- a.pop(0) 拿出 2，再 append 到尾巴：[3, 4, 2]。

第 8 題 答案：(A)

- 奇數放尾巴、偶數放前面：1 → 2 1 → 2 1 3 → 4 2 1 3 → 4 2 1 3 5。
- pop() 從尾巴拿掉 5，剩 4 2 1 3。

【錯】deque 的 pop() 是拿尾巴，popleft() 才是拿前面。

第 9 題 答案：(D)

- 出現次數：1 出現 2 次、2 出現 2 次、3 出現 3 次、5 出現 1 次。
- 次數最多的是 3（3 次），印出「數字 次數」：3 3。

第 10 題 答案：(C)

- 每個數只在「第一次出現」時印出，之後再出現就跳過。
- 所以保留原本出現的順序：4、7、1、9（不是由小到大）。

第 11 題 答案：(C)

- banana 裡 a 出現 3 次；Counter 查沒出現過的 z，得到 0，不會出錯。
- len(c) 是「不同字母」的個數：b、a、n 共 3 種。

【錯】一般 dict 查不存在的 key 才會 KeyError，Counter 不會。

第 12 題 答案：(B)

- 只有一種括號時，d 就是「堆疊裡有幾個左括號」。
- d 依序是 1 2 1 2 3 2 1 0 1 0：最深 3 層，最後 d = 0（全部配對成功）。

第 13 題 答案：(D)

- 最近看過的最先被拿出來，就是「後進先出」。
- 佇列是先進先出，會回到「最早」看的頁面。

Part B 程式實作參考解答

實作 1 括號檢查器

解題思路

- 左括號放進堆疊，堆疊大小的最大值就是最深層數。
- 右括號：堆疊是空的、或頂端不是同一種左括號 → 不合法；否則拿掉頂端。
- 全部讀完後，堆疊必須是空的才合法。

【錯】只數左右括號個數相不相等：([)] 個數相等卻不合法，一定要用堆疊比對種類。

【錯】忘了最後檢查堆疊是否為空：(這種只有左括號的會被誤判成合法。

Python 括號檢查器

```python
s = input().strip()
pair = {")": "(", "]": "[", "}": "{"}
st = []
depth = 0
ok = True
for c in s:
    if c in "([{":
        st.append(c)
        depth = max(depth, len(st))
    elif not st or st[-1] != pair[c]:
        ok = False
        break
    else:
        st.pop()
print(f"Yes {depth}" if ok and not st else "No")
```

時間複雜度：O(|S|)

實作 2 後序計算機

解題思路

- 讀到數字：放進堆疊。
- 讀到運算符號：先拿出 b（右邊）、再拿出 a（左邊），算 a op b 放回去。
- 全部處理完，堆疊裡剩下的唯一一個數就是答案。

【錯】先拿出來的是右邊：2 10 - 要算 2 - 10 = -8，寫反會得到 8。

Python 後序計算機

```python
import sys
data = sys.stdin.read().split()
n = int(data[0])
st = []
for t in data[1:1 + n]:
    if t in ("+", "-", "*"):
        b = st.pop()   # 右邊
        a = st.pop()   # 左邊
        if t == "+":
            st.append(a + b)
        elif t == "-":
            st.append(a - b)
        else:
            st.append(a * b)
    else:
        st.append(int(t))
print(st[-1])
```

時間複雜度：O(n)

實作 3 櫃台叫號系統

解題思路

- 前後都要放、都要拿，正好是 deque（雙端佇列）：IN 放後面、VIP 放前面、OUT 拿前面、LEAVE 拿後面。
- OUT 和 LEAVE 之前都要先檢查隊伍是不是空的。
- 輸出很多行，先收集在串列裡，最後用 join 一次印出。

【錯】用 list 的 insert(0, x) / pop(0) 模擬：每次 O(n)，2×10^5 個指令會超時。

【錯】隊伍空的時候 LEAVE 什麼都不做，不要輸出任何東西。

Python 櫃台叫號系統

```python
import sys
from collections import deque
data = sys.stdin.read().split()
m = int(data[0])
dq = deque()
out = []
i = 1
for _ in range(m):
    cmd = data[i]
    i += 1
    if cmd == "IN":
        dq.append(data[i])
        i += 1
    elif cmd == "VIP":
        dq.appendleft(data[i])
        i += 1
    elif cmd == "OUT":
        out.append(dq.popleft() if dq else "EMPTY")
    elif dq:
        dq.pop()   # LEAVE
out.append(str(len(dq)))
print("\n".join(out))
```

時間複雜度：O(m)，每個指令 O(1)

實作 4 急診叫號

解題思路

- 每次都要「目前最大」→ 優先佇列，放入、拿出都是 O(log n)。
- 同嚴重度先到先看：把「到達順序」也放進去一起比較。
- Python 的 heapq 小的先出：放 (-嚴重度, 到達順序, 名字)，嚴重度取負號就變成大的先出。

【錯】忘了取負號：heapq 會先叫出嚴重度最低的病人。

【錯】每次 CALL 都把等候名單整個排序：O(m^2 log m)，一定超時。

Python 急診叫號

```python
import sys
import heapq
data = sys.stdin.read().split()
m = int(data[0])
h, out, i, order = [], [], 1, 0
for _ in range(m):
    if data[i] == "ADD":
        name, v = data[i + 1], int(data[i + 2])
        heapq.heappush(h, (-v, order, name))   # 負號 → 大的先出
        order += 1
        i += 3
    else:
        out.append(heapq.heappop(h)[2] if h else "NONE")
        i += 1
print("\n".join(out))
```

時間複雜度：O(m log m)

實作 5 班代表選舉

解題思路

- 用 Counter 計數：一行就數完。
- sorted(cnt.items(), key=lambda t: (-t[1], t[0]))：先比負的票數，再比名字。

【錯】直接走訪 Counter 輸出：那是「第一次出現」的順序，不是題目要的順序。

【錯】同票時要比名字：ab 排在 abc 前面，abc 排在 b 前面（字典序逐字比較）。

Python 班代表選舉

```python
import sys
from collections import Counter
data = sys.stdin.read().split()
n = int(data[0])
cnt = Counter(data[1:1 + n])
items = sorted(cnt.items(), key=lambda t: (-t[1], t[0]))
print(len(items))
print("\n".join(f"{s} {c}" for s, c in items))
```

時間複雜度：O(n log n)

自我診斷表

對完答案，數一數每個觀念錯幾題：全對圈綠、錯 1 題圈黃、錯 2 題以上圈紅。紅燈的觀念回去看講義那一段。

觀念	題號	錯幾題	燈號（圈一個）	回去看哪裡
堆疊操作	A1、A5		綠 黃 紅	講義第 1 段
括號配對	A12、B1		綠 黃 紅	講義第 1 段「括號配對」
後序運算式	A3、A4、B2		綠 黃 紅	講義第 2 段（先拿出來的是右邊）
佇列模擬	A2、A8、B3		綠 黃 紅	講義第 3 段
優先佇列	A6、B4		綠 黃 紅	講義第 4 段
計數與去重	A9、A10、A11、B5		綠 黃 紅	講義第 5 段
陣列與 list 模擬	A7		綠 黃 紅	講義第 6 段
資料結構選擇	A13		綠 黃 紅	講義第 1～5 段：後進先出、先進先出、最優先